Model Checking Two Concurrent Programs with SPIN

RW796 Software Verification and Analysis

Gabriella **[TODO: surname]**
[TODO: student number]

August 29, 2026

Introduction

[TODO: One paragraph: which two programs were modelled and what the analysis found.]

Implementation under analysis: **[TODO: repo URL]**
Promela models, properties and verification records: **[TODO: repo URL]**

1 Program 1: FCFS Process Scheduler

1.1 The Promela Model

[TODO: Model description.]

Table 1: Mapping between the implementation and the model.

`manager.c`	**Model**	**Reason**
Thread in `schedule_fcfs`	`active [NUM_WORKERS] proctype Worker`	**[TODO:]**
`readyq`, `waitingq`	buffered channels	**[TODO:]**
`terminatedq`	`pcb_state[] == TERMINATED`	**[TODO:]**
`pcb_t.state`	`pcb_state[]`	**[TODO:]**
`next_instruction`	`instr_count[]`	**[TODO:]**
`resource_t.allocated`	`resource_owner[]`	**[TODO:]**
`q_locks[0]`, `omp critical`	`atomic` block	**[TODO:]**
`terminate()`	`check_terminate` inline	**[TODO:]**

Listing 1: Dequeue in `schedule_fcfs`

```
#pragma omp critical
{
  omp_set_nest_lock(&q_locks[0]);
  running_p = dequeue_pcb(&readyq);
  omp_unset_nest_lock(&q_locks[0]);
}
```

Listing 2: The same step in the model

```
\todo{paste the atomic dequeue block from model.pml}
```

1.2 Correctness Properties

`pcb_mutex`: mutual exclusion over process control blocks

$$\Box \bigwedge_{p=1}^{3} executing[p] \leq 1$$

```
#define one_worker_per_pcb (executing[1] <= 1 && executing[2] <= 1 &&
    executing[3] <= 1)
ltl pcb_mutex { [] one_worker_per_pcb }
```

[TODO: Justification.]

`no_self_wait`: no process waits for a resource it owns

$$\Box \bigwedge_{p=1}^{3} \big(waiting\_for[p] = 0 \vee resource\_owner[waiting\_for[p]] \neq p\big)$$

[TODO: Promela form and justification.]

`waiting_consistent`: the waiting state is consistent

[TODO: Formula, Promela form and justification.]

`no_dup_ready`: ready queue integrity

[TODO: Formula, Promela form and justification.]

`all_terminate`: every process eventually terminates

$$\Diamond \bigwedge_{p=1}^{3} pcb\_state[p] = TERMINATED$$

[TODO: Justification.]

`ready_dispatched`: a ready process is eventually dispatched

$$\Box\big(pcb\_state[1] = READY \rightarrow \Diamond\, pcb\_state[1] = RUNNING\big)$$

[TODO: Justification.]

Deadlock

[TODO: Discussion.]

1.3 Verification Results and Analysis

[TODO: Setup paragraph.]

The double dispatch race

[TODO: Analysis and fix.]

Self deadlock in `request_resource`

[TODO: Analysis and fix.]

Table 2: Verification results, Spin 6.5.2. Depths and state counts as reported by `pan`.

Property	Config	Result	Depth	States
`pcb_mutex`	`MAX_INSTR 3`	violated	901	65 957
`no_self_wait`	`MAX_INSTR 3`	violated	379	188
`waiting_consistent`	`MAX_INSTR 3`	violated	962	69 699
`no_dup_ready`	`MAX_INSTR 2`	holds	n/a	20 861 451
`all_terminate`	`MAX_INSTR 3, -f`	violated	1069	493
`ready_dispatched`	`MAX_INSTR 3, -f`	violated	1088	11 372 991
deadlock	`MAX_INSTR 3, -DNOCLAIM`	violated	504	67 872

Liveness and fairness

[TODO: Analysis.]

The invalid end state

[TODO: Analysis.]

The property that holds

[TODO: Analysis.]

2 Program 2: [TODO: name]

2.1 The Promela Model

[TODO:]

2.2 Correctness Properties

[TODO:]

2.3 Verification Results and Analysis

[TODO:]

A Reproducing the verification results

Listing 3: Build

```
spin -a model.pml
gcc -O2 -o pan pan.c
gcc -O2 -DNOCLAIM -o pan_noclaim pan.c
```

Listing 4: Verification runs

```
./pan -a -N pcb_mutex
./pan -a -N no_self_wait
./pan -a -N waiting_consistent
./pan -a -N all_terminate    -f -m50000
./pan -a -N ready_dispatched -f -m50000 -w26
./pan_noclaim
```

```
spin -a -DMAX_INSTR=2 model.pml
gcc -O2 -o pan pan.c
./pan -a -N no_dup_ready -m20000 -w26
```

[TODO: State the commit hash the results were produced from.]

References